



Laboratorio No. 1 Semana #13 – APIS con swagger & postman

Nombre: Alejandro Mogollón Rodriguez **Carné:** 1132125

Objetivo: Comprender y consumir una API REST mediante Swagger UI y Postman, identificando endpoints, métodos HTTP, parámetros, cuerpos JSON, códigos de respuesta y operaciones CRUD. El estudiante deberá interpretar la documentación de una API, ejecutar solicitudes y resolver un reto práctico utilizando evidencia técnica.

Investigación previa:

1. ¿Qué es una API REST?
Es un conjunto de reglas que permite que diferentes aplicaciones de software se comuniquen entre sí a través de internet usando el protocolo HTTP.
2. ¿Qué es un endpoint?
Es una dirección digital (URL) en programación o un dispositivo físico en redes que permite la comunicación entre sistemas.
3. ¿Qué diferencia existe entre una URL y un endpoint?
La URL es solo la ruta o dirección física o lógica. El endpoint incluye la URL más la regla o método de acción que el servidor espera recibir.
4. ¿Qué función cumplen los métodos HTTP GET, POST, PUT/PATCH y DELETE?
Indican la acción específica que un cliente (como el navegador) le pide realizar a un servidor sobre un recurso en la web.
5. ¿Qué es JSON y por qué es común en las APIs?
Es un formato de texto ligero diseñado para almacenar e intercambiar datos estructurados de forma clara y fácil de leer tanto para personas como para máquinas
6. ¿Qué es un código de estado HTTP? Mencione ejemplos de 200, 404 y 500.
Es un número de tres dígitos que el servidor envía como respuesta a una solicitud realizada por el navegador o cliente web, indicando el resultado de dicha petición.
 - 200 OK: Indica que la solicitud del cliente se ha completado.
 - 404 Not Found: Significa que el servidor no ha podido encontrar el recurso.
 - 500 Internal Server Error: Representa un error genérico del servidor.
7. ¿Qué información se puede encontrar en la respuesta de una API?
La línea de estado HTTP, encabezados con metadatos o información técnica sobre la respuesta y servidor y el body con la información principal que se estaba solicitando.
8. ¿Qué ventajas tiene utilizar Postman para probar una API antes de programarla?
 - Detectar inconsistencias en la arquitectura de datos o en el flujo de peticiones
 - Permite escribir scripts de prueba antes de programar el endpoint.
9. ¿Qué es y para qué sirve Swagger UI?
Es una herramienta de código abierto que genera una interfaz web interactiva para visualizar y probar la documentación de una API basada en la especificación OpenAPI.
10. ¿Qué función cumple Postman en el desarrollo y prueba de APIs?
Permite diseñar, enviar peticiones, probar, documentar y compartir APIs de forma visual y sin necesidad de escribir código complejo de cliente.



1. Preparación del entorno

Instalar o utilizar Postman: <https://www.postman.com/downloads/>

API a explorar, Swagger Petstore 3: <https://petstore3.swagger.io/>

Importante: La actividad se realizará mediante solicitudes HTTP GET y análisis de las respuestas JSON. No se requiere API Key para esta práctica.

2. Exploración inicial

Ingresa a Swagger Petstore y localice las operaciones relacionadas con /pet

- Identifique un endpoint que permita consultar una mascota por ID.
- Indique el método HTTP utilizado.
- Indique qué parámetro necesita la solicitud.
- Ejecute la solicitud con Try it out y registre el código HTTP recibido.
- Explique con sus palabras qué información contiene la respuesta.

Evidencia: captura de Swagger mostrando la operación y su respuesta.

3. Crear un recurso con POST

Utilice POST /pet para crear una mascota. Construya un JSON propio; no copie exactamente el ejemplo de Swagger.

El JSON debe incluir, como mínimo:

id
name
status
category
tags

Registre el ID utilizado y verifique la respuesta. Luego consulte nuevamente la mascota por medio de GET /pet/{petId}.

Evidencia: JSON enviado + respuesta de creación + consulta posterior.

4. Modificar el recurso

- Modifique la mascota creada utilizando PUT /pet. Cambie al menos dos atributos, por ejemplo nombre y estado.
- ¿Qué método HTTP utilizó y por qué?
- ¿Qué datos fueron enviados en el body?



- ¿Qué código HTTP devolvió la API?
- ¿Cómo comprobó que el cambio realmente ocurrió?

5. Consultar utilizando parámetros

- Utilice GET /pet/findByStatus y consulte las mascotas con estado available.
- Identifique el parámetro utilizado.
- Explique dónde se envía ese parámetro.
- Observe la estructura de la respuesta.
- Indique qué sucede si cambia el valor del parámetro por pending.

6. Reproducir las operaciones en Postman

- Ahora reproduzca en Postman las operaciones realizadas en Swagger. Cree una colección llamada 'MIA - Semana 13 - API'.
- La colección debe contener, como mínimo:
- GET – Consultar mascota por ID
- POST – Crear mascota
- PUT – Modificar mascota
- GET – Buscar por status
- DELETE – Eliminar mascota
- En cada solicitud configure manualmente método, URL, parámetros, headers y body cuando corresponda. No utilice únicamente la opción de importar una colección.

7. Prueba de errores

Una API también debe analizarse cuando algo sale mal. Realice al menos tres pruebas que generen respuestas diferentes a una solicitud exitosa.

Ejemplos de escenarios:

1. Consultar un ID inexistente.
2. Enviar un JSON incorrecto o incompleto.
3. Utilizar un parámetro con un valor no esperado.
4. Intentar eliminar un recurso que ya no existe.

Para cada prueba registre:

- a) Solicitud realizada.
- b) Código HTTP recibido.
- c) Respuesta de la API.
- d) Qué esperaba que ocurriera.
- e) Qué ocurrió realmente.
- f) Explicación de la causa

8. API Detective

Suponga que usted recibe el siguiente requerimiento de un sistema:

“El sistema debe registrar una mascota, consultar su información, actualizar su estado y finalmente eliminarla. Si



Resuelva el reto:

- Defina qué endpoint utilizaría para cada operación.
- Defina el método HTTP correspondiente.
- Indique qué respuesta debe analizar el programa.
- Ejecute el flujo completo en Postman.

Capture evidencia de cada paso.

Entrega

Dentro del repositorio de GitHub, cree la carpeta semana-13/lab1/ y coloque en ella un PDF con la solución del laboratorio.

Incluya también los archivos de respaldo generados durante la práctica, así como los pantallazos que evidencien las pruebas realizadas.

Enviar link de github a plataforma de la universidad.

The screenshot shows a web browser displaying the Swagger Petstore API documentation at <https://petstore3.swagger.io/api/v3/openapi.json>. The page title is "Swagger Petstore - OpenAPI 3.0". Below the title, there is a list of endpoints and their methods:

- PUT** /pet Update an existing pet.
- POST** /pet Add a new pet to the store.
- GET** /pet/findById/status Finds Pets by status.
- GET** /pet/findById/tags Finds Pets by tags.
- GET** /pet/{petId} Find pet by ID.
- POST** /pet/{petId} Updates a pet in the store with form data.
- DELETE** /pet/{petId} Deletes a pet.
- POST** /pet/{petId}/uploadImage Uploads an image.

On the right side of the browser, the Chrome DevTools Network tab is open, showing a list of requests. The first request is from petstore3.swagger.io to <https://petstore3.swagger.io/api/v3/openapi.json> using the GET method. The status is 200 OK. The response headers are visible, including Content-Type: application/json and Date: Wed, 30 Sep 2026 13:36:51 GMT.

[illegible]



amogollon-1-s-team.postman.co/workspace/MIA_2026~78e596e6-b59b-4925-977f-b4d5e16e2f76/http-request/...

Postman interface showing a GET request to `https://postman-echo.com/get?foo1=bar1&foo2=bar2`. The request is configured with query parameters `foo1=bar1` and `foo2=bar2`. The response is a 200 OK status, 503 ms, 1.22 KB, with a JSON body containing the request details.

Query Params

Key	Value	Description	Bulk Edit
foo1	bar1		
foo2	bar2		

Body

```
1 {
2   "args": {
3     "foo1": "bar1",
4     "foo2": "bar2"
5   },
6   "headers": {
7     "host": "postman-echo.com",
8     "x-forwarded-proto": "https",
9     "postman-token": "469fe3df-ice7-4e13-9f0a-c9284b9bbf81",
10    "user-agent": "PostmanRuntime/2.10.0",
11    "accept-encoding": "gzip, br",
12    "accept": "*/*",
13    "cache-control": "no-cache"
14  }
15 }
```